



Cobertura

Para testes unitários



100% de Cobertura

- Representa a execução de todas as linhas de código
- Captura erros em cálculos, em comparações e em fluxos de controle:
 - É fundamental testar as fronteiras
 - Exemplo: valores -1, 0, n e (n+1) para um fluxo de controle de 0 a n
- 100% de cobertura é geralmente inviável:
 - Envolve testar todos os caminhos possíveis de execução
 - Tempo e custo podem inviabilizar
 - O ideal é focar em pontos críticos!



Projeto de testes

- Conjunto Básico de Testes:
 - Exercitar ao menos uma vez todas as sentenças de um programa
 - Se baseia no grafo de fluxo do programa e na sua complexidade ciclomática
 - Permite identificar os caminhos independentes de teste, e um conjunto mínimo destes
- Complexidade ciclomática:
 - Quantidade de caminhas independentes de um código
 - Geram caminhos independentes:
 - Estruturas de controle (if-else, switch)
 - Estruturas de Repetição (while, do-while, for)

Complexidade Ciclomática

Procedimento média(valor[])

i=1;

soma=0;

total.entrada=0;

total.válidas=0

Faça-Enquanto (valor[i]≠-999 **E** total.entrada<100)

 incremente total.entrada de 1;

Se (valor[i]≥min **E** valor[i]≤max)

Então

 incremente total.válidas de 1;
 soma = soma + valor[i]

Fim-Se

 incremente i de 1;

Fim-Enquanto

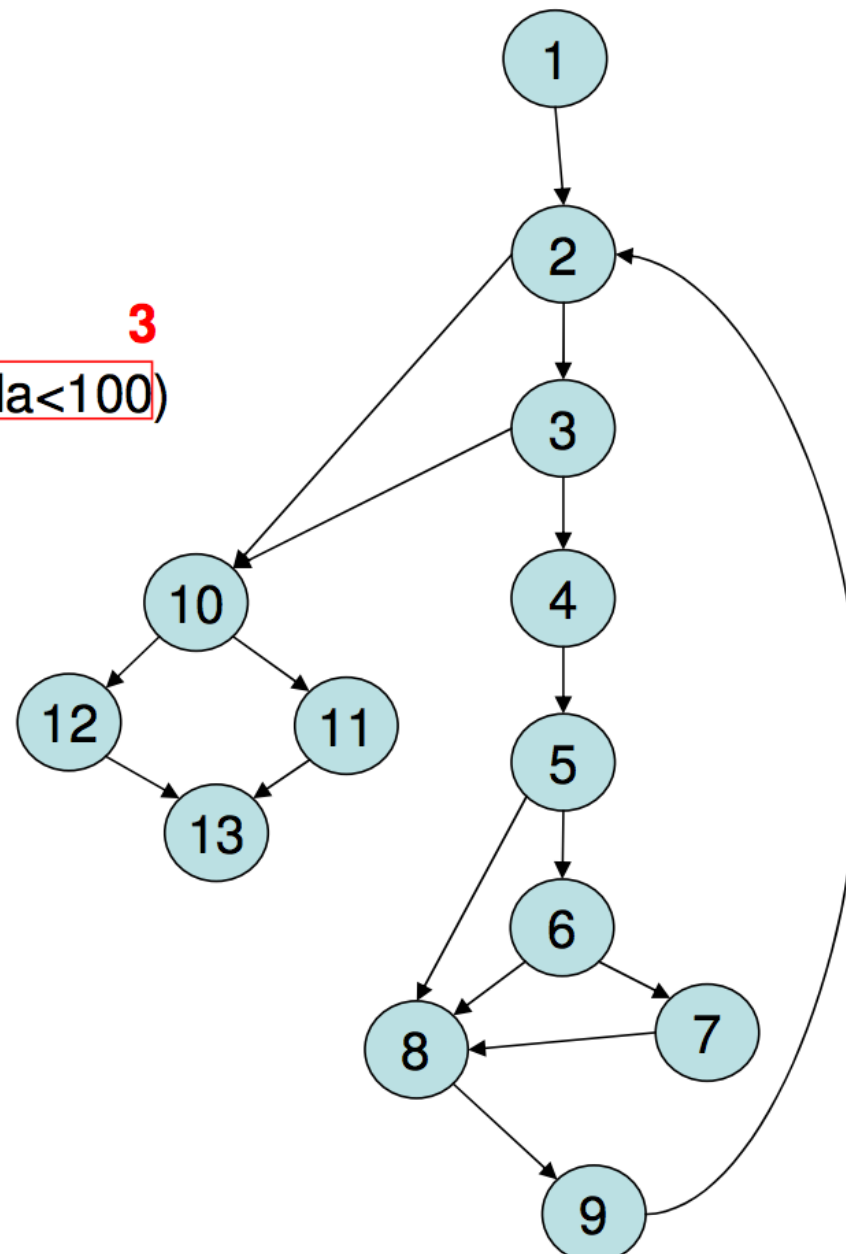
Se total.válidas>0

Então média = soma/total.válidas;

Senão média = -999;

Fim-Se

Fim média



Complexidade Ciclomática

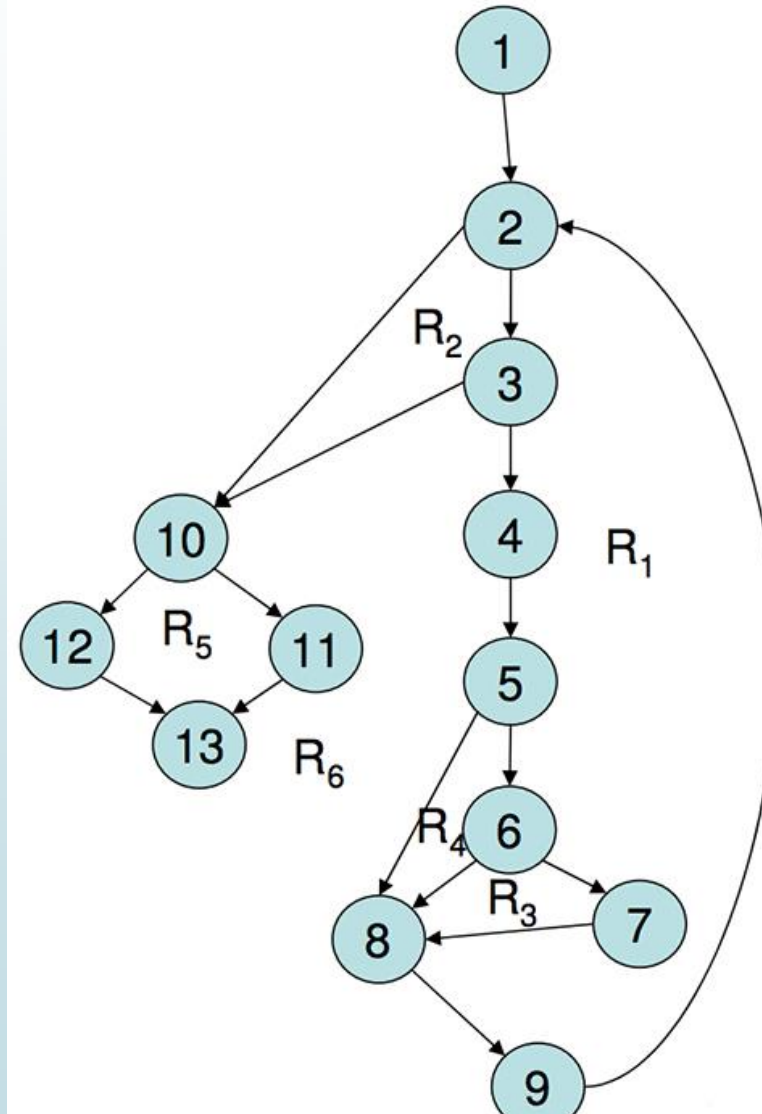
► 3 Formas de calcular:

1. $V(G) = R$

onde R é o número de regiões do grafo de fluxo

No exemplo:

$V(G) = 6$ regiões



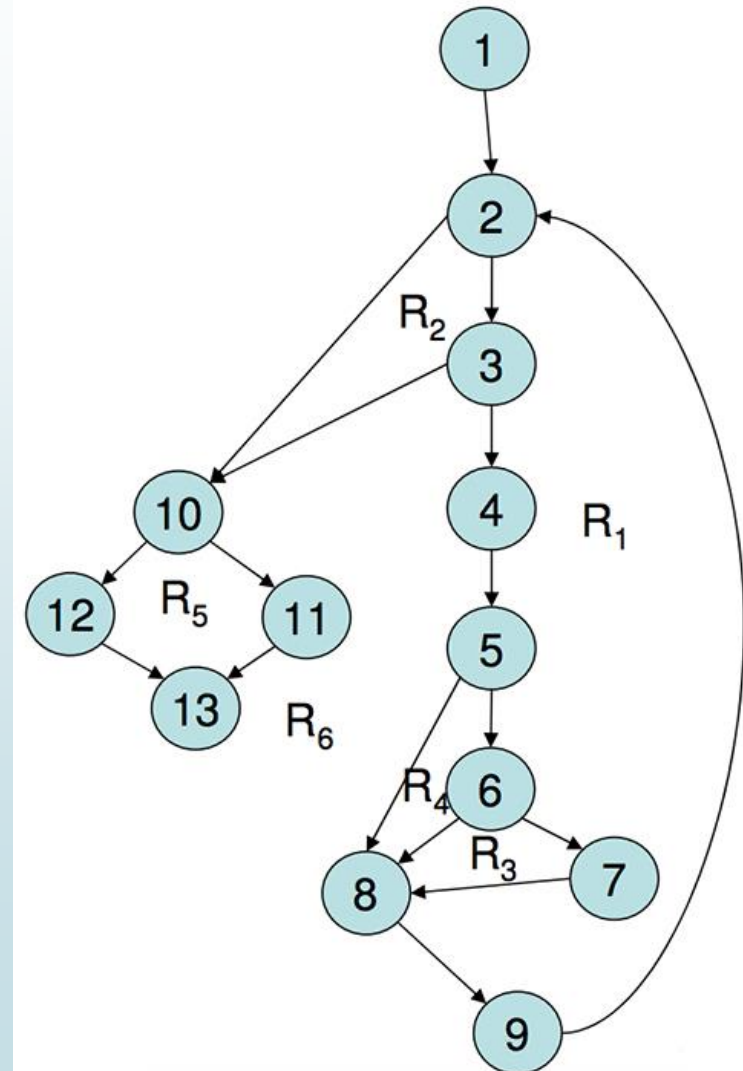
Complexidade Ciclomática

► 3 Formas de calcular:

2. $V(G) = E - N + 2$

onde E é o número de arestas (setas)
e N é o número de nós do grafo G.

► $V(G) = 17 \text{ arestas/setas}$
– $13 \text{ nós} + 2 = 6$



Complexidade Ciclomática

► 3 Formas de calcular:

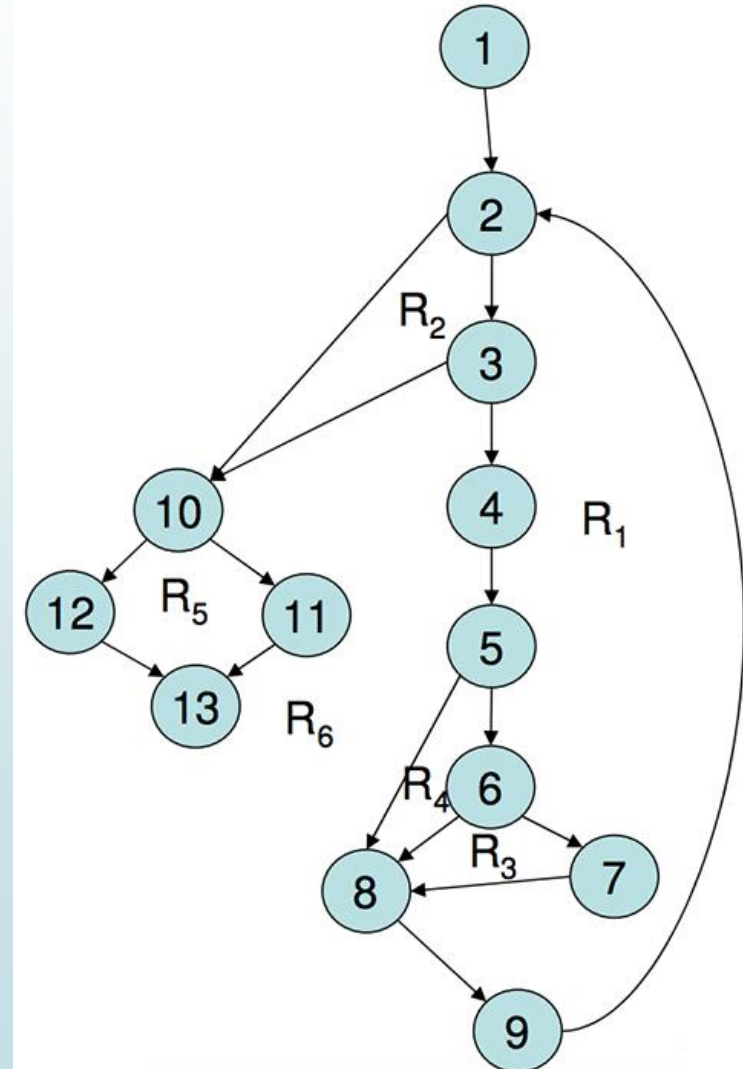
3. $V(G) = P + 1$

onde P é o número de nós-predicado do grafo G .

► $V(G) = 5 \text{ nós-predicado} + 1 = 6$

Nós-predicado são àqueles que podem desviar o fluxo da execução: *if*, *while*, *switch* etc.

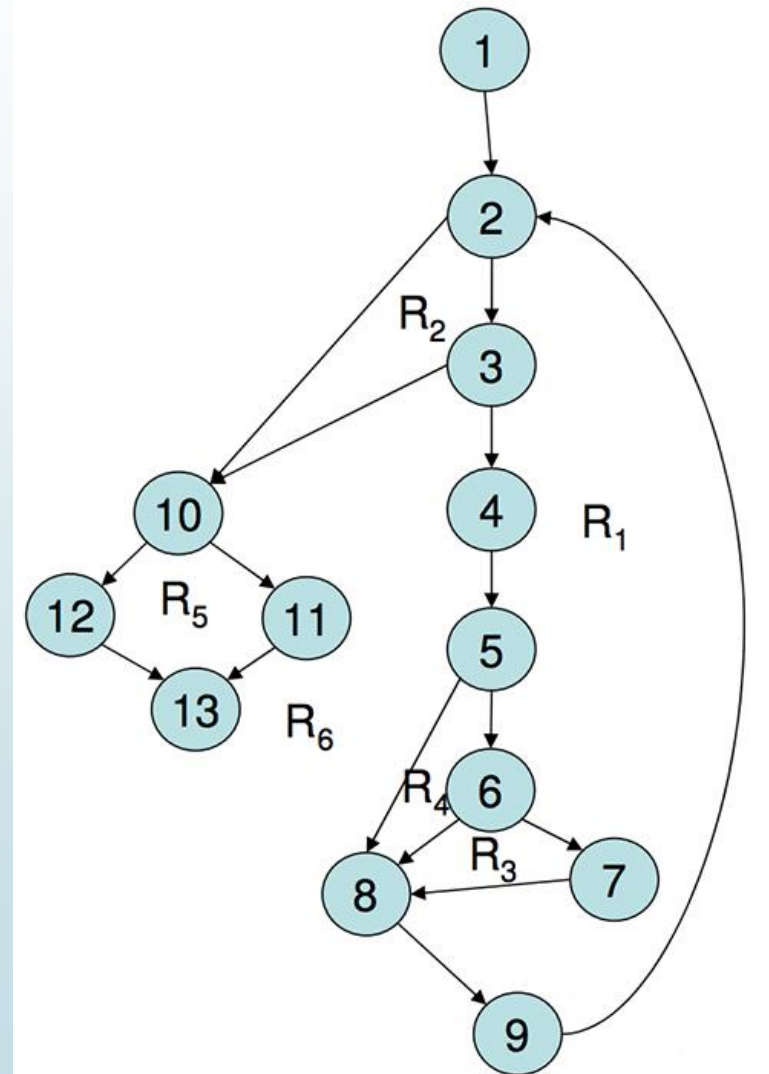
(Precisa ter 2 arestas no máximo saindo deles)



Complexidade Ciclomática

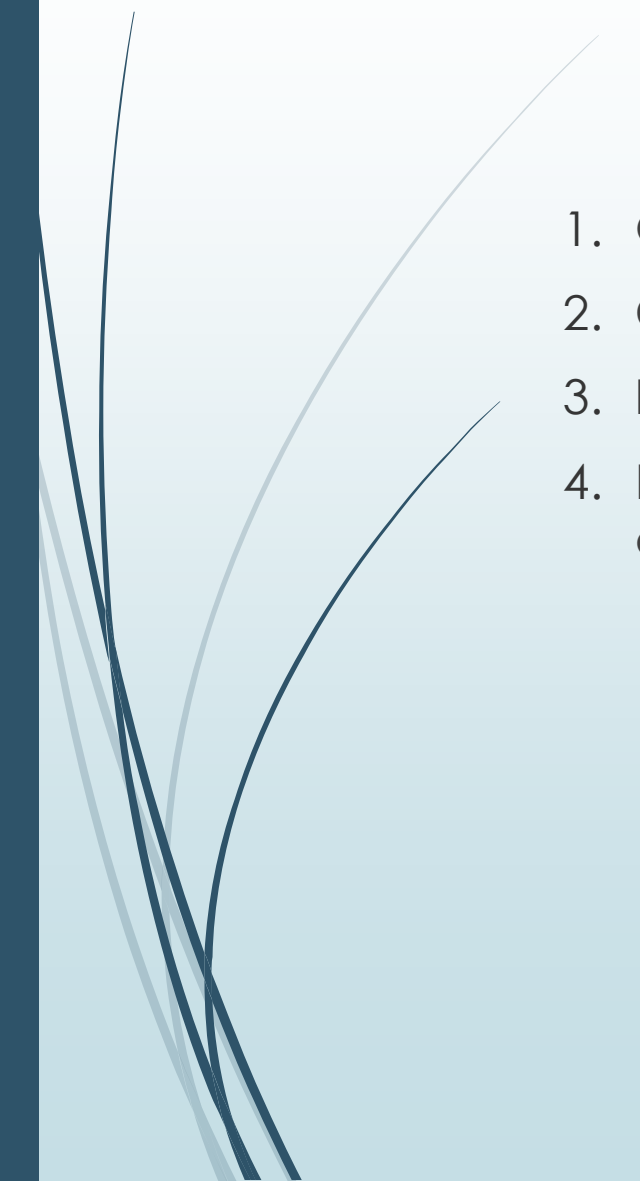
► Os 6 caminhos independentes são:

- **1)** 1-2-10-12-13
- **2)** 1-2-10-11-13
- **3)** 1-2-3-10-11-13
- **4)** 1-2-3-4-5-8-9-2- [...]
- **5)** 1-2-3-4-5-6-8-9-2- [...]
- **6)** 1-2-3-4-5-6-7-8-9-2- [...]





Projeto de Testes



1. Gerar o gráfico de fluxo para cada método/procedimento/função
2. Calcular a complexidade ciclomática do grafo de fluxo
3. Identificar os caminhos independentes
4. Elaborar casos de teste com valores de entrada que exercitem cada um dos caminhos independentes

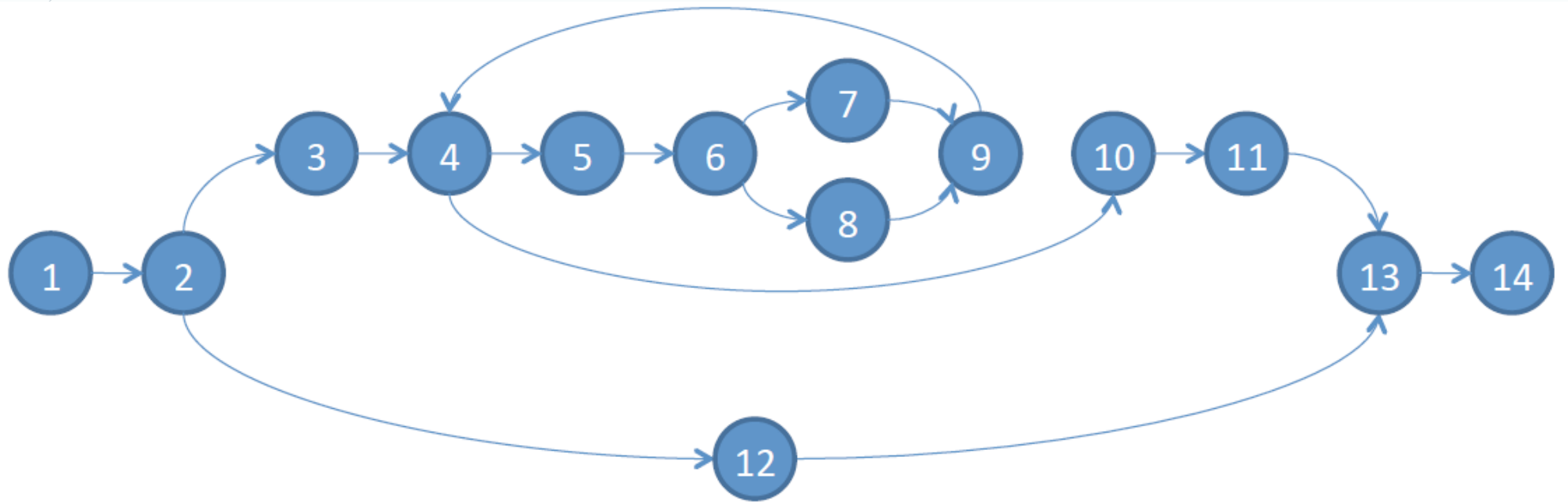
Exemplo (quicksort)

```
public List<String> ordena(List<String> listaDesordenada) {  
    List<String> listaOrdenada = new ArrayList<String>();  
    if (listaDesordenada.size() > 1) {  
        String pivo = listaDesordenada.get(0);  
        List<String> listaMenoresDesordenada = new ArrayList<String>();  
        List<String> listaMaioresDesordenada = new ArrayList<String>();  
        for (int i = 1; i < listaDesordenada.size(); i++) {  
            String elemento = listaDesordenada.get(i);  
            if (elemento.compareTo(pivo) < 0) { listaMenoresDesordenada.add(elemento); }  
            else { listaMaioresDesordenada.add(elemento); }  
        }  
        listaOrdenada.addAll(ordena(listaMenoresDesordenada));  
        listaOrdenada.add(pivo);  
        listaOrdenada.addAll(ordena(listaMaioresDesordenada));  
    } else { listaOrdenada.addAll(listaDesordenada); }  
    return listaOrdenada;  
}
```

Exemplo (quicksort)

```
public List<String> ordena(List<String> listaDesordenada) {  
  1 List<String> listaOrdenada = new ArrayList<String>();  
  2 if (listaDesordenada.size() > 1) {  
    String pivo = listaDesordenada.get(0);  
    3 List<String> listaMenoresDesordenada = new ArrayList<String>();  
    List<String> listaMaioresDesordenada = new ArrayList<String>();  
    4 for (int i = 1; i < listaDesordenada.size(); i++) {  
      5 String elemento = listaDesordenada.get(i);  
      6 if (elemento.compareTo(pivo) < 0) { listaMenoresDesordenada.add(elemento) } 7  
      8 else { listaMaioresDesordenada.add(elemento); } 9  
    10 }  
    11 listaOrdenada.addAll(ordena(listaMenoresDesordenada));  
    listaOrdenada.add(pivo);  
    listaOrdenada.addAll(ordena(listaMaioresDesordenada));  
  12 } else { listaOrdenada.addAll(listaDesordenada); } 13  
  14 return listaOrdenada;  
}
```

Grafo de fluxo e Complexidade Ciclomática



- $V(G) = E - N + 2 = 16 - 14 + 2 = 4$
 - Teste 1: 1, 2, 12, 13, 14
 - Teste 2: 1, 2, 3, 4, 10, 11, 13, 14 (IMPOSSÍVEL)
 - Teste 3: 1, 2, 3, 4, 5, 6, 7, 9, 4, 10, 11, 13, 14
 - Teste 4: 1, 2, 3, 4, 5, 6, 8, 9, 4, 10, 11, 13, 14

Exemplo – Teste 1 com JUnit

```
@Test
public void teste1() {
    List<String> listaDesordenada = Arrays.asList("abc");
    List<String> oraculo = Arrays.asList("abc");

    List<String> resultado = quicksort.ordena(listaDesordenada);
    assertEquals(oraculo, resultado);
}
```

Exemplo – Teste 2 com JUnit

```
@Test
public void teste2() {
    List<String> listaDesordenada = Arrays.asList("def", "abc");
    List<String> oraculo = Arrays.asList("abc", "def");

    List<String> resultado = quicksort.ordena(listaDesordenada);
    assertEquals(oraculo, resultado);
}
```

Exemplo – Teste 3 com JUnit

```
@Test
public void teste3() {
    List<String> listaDesordenada = Arrays.asList("abc", "def");
    List<String> oraculo = Arrays.asList("abc", "def");

    List<String> resultado = quicksort.ordena(listaDesordenada);
    assertEquals(oraculo, resultado);
}
```

Exemplo (quicksort com defeito)

```
public List<String> ordena(List<String> listaDesordenada) {  
    List<String> listaOrdenada = new ArrayList<String>();  
    if (listaDesordenada.size() > 1) {  
        String pivo = listaDesordenada.get(0);  
        List<String> listaMenoresDesordenada = new ArrayList<String>();  
        List<String> listaMaioresDesordenada = new ArrayList<String>();  
        for (int i = 1; i < listaDesordenada.size() - 1; i++) {  
            String elemento = listaDesordenada.get(i);  
            if (elemento.compareTo(pivo) < 0) { listaMenoresDesordenada.add(elemento) }  
            else { listaMaioresDesordenada.add(elemento); }  
        }  
        listaOrdenada.addAll(ordena(listaMenoresDesordenada));  
        listaOrdenada.add(pivo);  
        listaOrdenada.addAll(ordena(listaMaioresDesordenada));  
    } else { listaOrdenada.addAll(listaDesordenada); }  
    return listaOrdenada;  
}
```

Exemplo (resultado JUnit)

